

Java Swing

Components, Layout Managers, Forms, Menus and the Development of a Small Desktop Application

Name	Kishore G
Roll No.	26MCT010
Programme	M.Sc Computer Technology
Department	Information Technology and Cognitive Systems
College	Sri Krishna College of Arts and Science

1. Introduction

Java Swing is the part of the Java Foundation Classes that is used to build graphical desktop applications. It came after the older AWT (Abstract Window Toolkit) and was designed to fix most of the problems AWT had. AWT depended on the underlying operating system to draw its components, which meant a button on Windows could look and behave slightly differently from the same button on Linux or macOS. Swing components, on the other hand, are drawn entirely by Java itself using a system called "lightweight rendering". This makes a Swing application look and behave the same way regardless of which operating system it is running on.

This document walks through the main building blocks of Swing programming: the common components you will use in almost every application, the layout managers that decide how those components are arranged on screen, how to put components together into a working form, how to add menus to a window, and finally how all of these pieces come together in a small but complete desktop application. Every section includes Java code so that the explanation is not just theoretical.

Before going further, it helps to know two classes that almost every Swing program depends on: **JFrame**, which represents the actual application window, and **JPanel**, which is a container used to group other components together. Nearly everything discussed below eventually gets placed inside one of these two.

2. Swing Components

A Swing component is simply a visual element that the user can see and, in most cases, interact with. Swing provides a large library of ready-made components, and the table below lists the ones that show up most often in everyday applications.

Component	Purpose
JLabel	Displays text or an icon; cannot be edited by the user
JButton	A clickable push button that triggers an action
JTextField	A single-line box for typing text
JTextArea	A multi-line box for typing larger amounts of text
JCheckBox	A box that can be ticked on or off, independent of other boxes
JRadioButton	A round button used in a group where only one can be selected
JComboBox	A drop-down list that lets the user pick one value
JList	A scrollable list that can allow single or multiple selection
JTable	Displays data in rows and columns, similar to a spreadsheet
JOptionPane	Shows quick pop-up dialogs for messages, input or confirmation

Rather than describing each component in isolation, it is more useful to see several of them working together inside one window. The example below creates a frame that contains a label, a text field, a combo box, a couple of checkboxes and a button. When the button is pressed, the values entered

by the user are read and displayed through a message dialog.

```
import javax.swing.*;
import java.awt.event.*;

public class ComponentDemo extends JFrame {

    private JTextField nameField;
    private JComboBox<String> courseBox;
    private JCheckBox hostelBox, scholarshipBox;

    public ComponentDemo() {
        setTitle("Swing Component Demo");
        setSize(380, 260);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new java.awt.FlowLayout());

        add(new JLabel("Student Name:"));
        nameField = new JTextField(15);
        add(nameField);

        add(new JLabel("Course:"));
        courseBox = new JComboBox<>(new String[] {
            "M.Sc Computer Technology", "B.Sc Computer Science", "BCA"
        });
        add(courseBox);

        hostelBox = new JCheckBox("Staying in hostel");
        scholarshipBox = new JCheckBox("Applied for scholarship");
        add(hostelBox);
        add(scholarshipBox);

        JButton submitButton = new JButton("Submit");
        submitButton.addActionListener(e -> showSummary());
        add(submitButton);
    }

    private void showSummary() {
        String message = "Name: " + nameField.getText()
            + "\nCourse: " + courseBox.getSelectedItem()
            + "\nHostel: " + hostelBox.isSelected()
            + "\nScholarship: " + scholarshipBox.isSelected();
        JOptionPane.showMessageDialog(this, message, "Summary",
            JOptionPane.INFORMATION_MESSAGE);
    }

    public static void main(String[] args) {
        new ComponentDemo().setVisible(true);
    }
}
```

Example 1: several Swing components combined in one window.

A few points are worth remembering here. First, almost every interactive component in Swing follows the same pattern: you create the component, add it to a container, and then attach a listener to it so that your code reacts when the user interacts with it. Second, **ActionListener** is the most common listener type, and since Java 8 it can be written as a short lambda expression instead of a full anonymous class, which is what the example above does for the submit button.

3. Layout Managers

Once you have more than one or two components, you need a way to decide where each one appears on the screen and how they resize when the window is resized. This is the job of a layout manager. Swing does not expect you to set pixel coordinates by hand; instead, you choose a layout manager for a container, and that manager decides the positioning automatically. Choosing the right layout manager is often the difference between a form that looks organised and one that looks scattered.

3.1 FlowLayout

This is the simplest layout. Components are placed left to right in the order they are added, and when a row fills up, the next component moves to a new row. It is the default layout for a plain JPanel and is useful for small groups of buttons or controls where exact alignment does not matter much.

```
JPanel panel = new JPanel(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT, 10, 10));
panel.add(new JButton("OK"));
panel.add(new JButton("Cancel"));
```

3.2 BorderLayout

BorderLayout divides the container into five regions: North, South, East, West and Center. This is the default layout for a JFrame's content pane. It is a natural fit for an application window where, for example, a toolbar sits at the top, a status bar sits at the bottom, and the main working area fills the centre.

```
setLayout(new java.awt.BorderLayout());
add(new JLabel("Toolbar area", SwingConstants.CENTER), java.awt.BorderLayout.NORTH);
add(new JTextArea(), java.awt.BorderLayout.CENTER);
add(new JLabel("Status: Ready"), java.awt.BorderLayout.SOUTH);
```

3.3 GridLayout

GridLayout arranges components in a fixed grid of rows and columns, and every cell in the grid is given exactly the same size. It works well for things like a calculator keypad, where every button should look identical in size.

```
JPanel keypad = new JPanel(new java.awt.GridLayout(4, 3, 5, 5));
for (int i = 1; i <= 9; i++) {
    keypad.add(new JButton(String.valueOf(i)));
}
keypad.add(new JButton("0"));
keypad.add(new JButton("Clear"));
keypad.add(new JButton("="));
```

3.4 GridBagLayout

GridBagLayout is the most flexible, and also the most detailed, layout manager in Swing. Components can span multiple rows or columns and can be given individual alignment and spacing rules through a helper object called GridBagConstraints. It takes more code to set up than the other layouts, but it is the standard choice when building forms where labels and input fields need to line up neatly in columns, which is exactly what the forms section below uses it for.

3.5 BoxLayout and CardLayout

BoxLayout arranges components strictly in a single row or a single column and is often used to stack a group of controls vertically with even spacing. CardLayout is different in that it does not display every component at once; instead, it stores several panels like a stack of cards and shows

only one at a time, which makes it useful for wizard-style screens or for switching between different views, such as a login panel and a main application panel, inside the same window.

4. Building Forms

A form is essentially a panel that gathers structured input from the user, usually made up of labels paired with text fields, combo boxes, checkboxes or radio buttons, and one or more buttons to submit or reset that input. The main design goal of a form is readability: labels and fields should line up so the user can scan down the form easily. This is where `GridBagLayout` is genuinely useful, since it lets each label sit next to its own field in a predictable column.

The example below builds a small student registration form. It also includes basic validation, since a form that accepts any input, including empty fields, is not very useful in a real application.

```
import javax.swing.*;
import java.awt.*;

public class RegistrationForm extends JFrame {

    private JTextField nameField, emailField;
    private JTextField ageField;
    private JRadioButton maleButton, femaleButton;

    public RegistrationForm() {
        setTitle("Student Registration");
        setSize(400, 260);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel formPanel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(6, 6, 6, 6);
        gbc.anchor = GridBagConstraints.WEST;

        gbc.gridx = 0; gbc.gridy = 0;
        formPanel.add(new JLabel("Full Name:"), gbc);
        gbc.gridx = 1;
        nameField = new JTextField(16);
        formPanel.add(nameField, gbc);

        gbc.gridx = 0; gbc.gridy = 1;
        formPanel.add(new JLabel("Email:"), gbc);
        gbc.gridx = 1;
        emailField = new JTextField(16);
        formPanel.add(emailField, gbc);

        gbc.gridx = 0; gbc.gridy = 2;
        formPanel.add(new JLabel("Age:"), gbc);
        gbc.gridx = 1;
        ageField = new JTextField(16);
        formPanel.add(ageField, gbc);

        gbc.gridx = 0; gbc.gridy = 3;
        formPanel.add(new JLabel("Gender:"), gbc);
        JPanel genderPanel = new JPanel();
        maleButton = new JRadioButton("Male");
        femaleButton = new JRadioButton("Female");
        ButtonGroup group = new ButtonGroup();
        group.add(maleButton);
        group.add(femaleButton);
        genderPanel.add(maleButton);
        genderPanel.add(femaleButton);
        gbc.gridx = 1;
```

```

        formPanel.add(genderPanel, gbc);

        JButton registerButton = new JButton("Register");
        registerButton.addActionListener(e -> validateAndSubmit());
        gbc.gridx = 1; gbc.gridy = 4;
        formPanel.add(registerButton, gbc);

        add(formPanel);
    }

    private void validateAndSubmit() {
        String name = nameField.getText().trim();
        String email = emailField.getText().trim();
        String age = ageField.getText().trim();

        if (name.isEmpty() || email.isEmpty() || age.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please fill in every field.",
                "Incomplete Form", JOptionPane.WARNING_MESSAGE);
            return;
        }
        if (!email.contains("@")) {
            JOptionPane.showMessageDialog(this, "Please enter a valid email address.",
                "Invalid Email", JOptionPane.WARNING_MESSAGE);
            return;
        }
        try {
            Integer.parseInt(age);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Age must be a number.",
                "Invalid Age", JOptionPane.WARNING_MESSAGE);
            return;
        }
        JOptionPane.showMessageDialog(this, "Registration successful for " + name,
            "Success", JOptionPane.INFORMATION_MESSAGE);
    }

    public static void main(String[] args) {
        new RegistrationForm().setVisible(true);
    }
}

```

Example 2: a registration form using GridBagLayout with input validation.

Notice that the validation happens before anything is treated as a successful submission. This is a habit worth carrying into any form you build: check that required fields are filled, check that the data is in the expected format, and only then proceed. Doing this consistently avoids a large number of bugs that otherwise only show up when a user types something unexpected.

5. Menus

Most desktop applications place their main commands inside a menu bar rather than scattering buttons everywhere. Swing builds menus from three classes working together: **JMenuBar** sits at the top of the frame and holds one or more **JMenu** objects, and each **JMenu** holds a list of **JMenuItem** objects that the user can click. Menu items can also be given keyboard shortcuts, called accelerators, so that frequent actions do not always require reaching for the mouse.

```

import javax.swing.*;
import java.awt.event.*;

public class MenuDemo extends JFrame {

    public MenuDemo() {

```

```

setTitle("Menu Demo");
setSize(400, 300);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

JMenuBar menuBar = new JMenuBar();

JMenu fileMenu = new JMenu("File");
JMenuItem newItem = new JMenuItem("New");
JMenuItem openItem = new JMenuItem("Open");
JMenuItem exitItem = new JMenuItem("Exit");

newItem.setAccelerator(KeyStroke.getKeyStroke("control N"));
exitItem.addActionListener(e -> System.exit(0));

fileMenu.add(newItem);
fileMenu.add(openItem);
fileMenu.addSeparator();
fileMenu.add(exitItem);

JMenu editMenu = new JMenu("Edit");
editMenu.add(new JMenuItem("Cut"));
editMenu.add(new JMenuItem("Copy"));
editMenu.add(new JMenuItem("Paste"));

menuBar.add(fileMenu);
menuBar.add(editMenu);
setJMenuBar(menuBar);

JTextArea textArea = new JTextArea();
add(textArea);

JPopupMenu popupMenu = new JPopupMenu();
popupMenu.add(new JMenuItem("Clear Text"));
textArea.setComponentPopupMenu(popupMenu);
}

public static void main(String[] args) {
    new MenuDemo().setVisible(true);
}
}

```

Example 3: a menu bar with File and Edit menus, plus a right-click popup menu.

The popup menu shown at the end of the example is attached directly to the text area and appears when the user right-clicks inside it. This is a small touch, but it is the kind of detail that makes an application feel complete rather than like a bare collection of components.

6. Development of a Small Desktop Application

To bring every idea discussed so far into one place, this section builds a small Student Record Manager. The application keeps a list of students in memory, displays them in a table, and lets the user add or delete records through a form. It uses BorderLayout for the overall window, GridLayout for the input form, a menu bar for application-level commands, and a JTable backed by a DefaultTableModel to show the data. Even though it is small, the structure here is close to what a real internal tool might look like.

```

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*.*;

public class StudentRecordManager extends JFrame {

```

```

private DefaultTableModel tableModel;
private JTable table;
private JTextField nameField, deptField;

public StudentRecordManager() {
    setTitle("Student Record Manager");
    setSize(500, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout(8, 8));

    setJMenuBar(buildMenuBar());
    add(buildFormPanel(), BorderLayout.NORTH);
    add(buildTablePanel(), BorderLayout.CENTER);
}

private JMenuBar buildMenuBar() {
    JMenuBar menuBar = new JMenuBar();
    JMenu fileMenu = new JMenu("File");
    JMenuItem exitItem = new JMenuItem("Exit");
    exitItem.addActionListener(e -> System.exit(0));
    fileMenu.add(exitItem);
    menuBar.add(fileMenu);
    return menuBar;
}

private JPanel buildFormPanel() {
    JPanel panel = new JPanel(new GridLayout(2, 3, 6, 6));
    panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 0, 10));

    nameField = new JTextField();
    deptField = new JTextField();

    panel.add(new JLabel("Student Name:"));
    panel.add(nameField);

    JButton addButton = new JButton("Add Record");
    addButton.addActionListener(e -> addRecord());
    panel.add(addButton);

    panel.add(new JLabel("Department:"));
    panel.add(deptField);

    JButton deleteButton = new JButton("Delete Selected");
    deleteButton.addActionListener(e -> deleteSelected());
    panel.add(deleteButton);

    return panel;
}

private JScrollPane buildTablePanel() {
    tableModel = new DefaultTableModel(new Object[] {"Name", "Department"}, 0);
    table = new JTable(tableModel);
    return new JScrollPane(table);
}

private void addRecord() {
    String name = nameField.getText().trim();
    String dept = deptField.getText().trim();
    if (name.isEmpty() || dept.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Both fields are required.");
        return;
    }
    tableModel.addRow(new Object[] {name, dept});
    nameField.setText("");
    deptField.setText("");
}

private void deleteSelected() {

```



```

        int row = table.getSelectedRow();
        if (row == -1) {
            JOptionPane.showMessageDialog(this, "Select a row to delete first.");
            return;
        }
        tableModel.removeRow(row);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new StudentRecordManager().setVisible(true));
    }
}

```

Example 4: a complete small application combining layouts, forms, a menu and a table.

A couple of design choices in this example are worth calling out because they are common in real Swing applications and not just in this demo. The window is broken into smaller private methods, one for the menu bar, one for the form, and one for the table, which keeps the constructor short and each piece easy to find. The table itself is driven by a **DefaultTableModel** rather than being filled directly, since the table model is what actually stores the rows and columns; the JTable component only displays whatever the model contains. Finally, the application is started inside **SwingUtilities.invokeLater**, which schedules the window creation on the Event Dispatch Thread. Swing is not thread-safe, so building the user interface on this specific thread is the recommended practice rather than an optional detail.

7. Conclusion

Swing programming, once broken into pieces, is fairly approachable. Components give you the visual building blocks, layout managers decide how those blocks are arranged, forms combine several components into a structured way of collecting input, and menus give the application a proper command structure instead of relying purely on buttons. The Student Record Manager example in the previous section shows that even a small application already touches every one of these ideas at once. Building slightly larger projects from here mainly means repeating the same patterns: containers inside containers, listeners attached to components, and a model object sitting behind anything that displays a list or a table of data.